

Hitec University Taxila, Department of Computer Engineering

Lab Manual for Mobile Application Development

Lab-2 Activity with GUI and Click Listener

Hitec University Taxila, Department of Computer Engineering	1
Lab Manual for Mobile Application Development	1
2. Activity Time boxing	3
3. Objectives of the experiment.....	3
4. Concept Map	3
4.1. Android UI Controls	4
4.2. Create UI Controls	5
4.3. Android Event Handling.....	6
5. Xml Files	7
5.1. Strings.xml	7
5.1.1. Location in Android Studio	7
6. Walk through Task	8
6.1. Creating New Activity	8
Step 1: Create New Project:	8
Step 2: Create New Activity.....	9
Step 3: Adding Button and Edit Text in Activity:	9
Step 4: Add View of layout in Activity:	10
Step 6: Add Button and Edit Text Reference in Activity:.....	10
Step 7: Add onClick Listener.....	11
Step 8: Run project and enter text and press button.	11
6.2. Practice Task 1	12

1. Introduction

The goal of this lab is to teach the skills necessary to create Activities in Android applications using the Android Studio Integrated Development Environment (IDE) and the Android Software Development Kit (SDK).

This lab will provide an outline of Android Activity with GUI. Activity is one of the buildingblocks of Android OS. In simple words Activity is a screen that user interact with. Every Activity in android has lifecycle like created, started, resumed, paused, stopped or destroyed. Basic GUI components are TextViews, Buttons, Image Views and Fragments.

2. Activity Time boxing

Table 1: Activity Time Boxing

Task No.	Activity Name	Activity time	Total Time
1	Introduction	60mins	60min
2	Walk Through Task	60mins	60min
3	Evaluation Task	60min	60min

3. Objectives of the experiment

- To get basic understanding regarding Android App Development
- To build a mobile app in android studio
- To understand the GUI components of Android

4. Concept Map

An Android application user interface is everything that the user can see and interact with. You have learned about the various layouts that you can use to position your views in an activity. This lab will give you detail on various views.

A **View** is an object that draws something on the screen that the user can interact with and a **View Group** is an object that holds other View (and View Group) objects in order to define the layout of the user interface. You define your layout in an XML file which offers a human-readable structure for the layout, similar to HTML. For example, a simple vertical layout with a text view and a button looks like this:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="vertical" >

    <TextView android:id="@+id/text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="I am a TextView" />

    <Button android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="I am a Button" />

/LinearLayout>
```

4.1. Android UI Controls

There are number of UI controls provided by Android that allow you to build the graphical user interface for your app

Sr. No.	UI Control & Description
1	TextView This control is used to display text to the user.
2	EditText EditText is a predefined subclass of TextView that includes rich editing capabilities.
3	AutoCompleteTextView The AutoCompleteTextView is a view that is similar to EditText, except that it shows a list of completion suggestions automatically while the user is typing.
4	Button A push-button that can be pressed, or clicked, by the user to perform an action.
5	ImageButton AbsoluteLayout enables you to specify the exact location of its children.

6	CheckBox An on/off switch that can be toggled by the user. You should use checkboxes when presenting users with a group of selectable options that are not mutually exclusive.
7	ToggleButton An on/off button with a light indicator.
8	RadioButton The RadioButton has two states: either checked or unchecked.
9	RadioGroup A RadioGroup is used to group together
10	ProgressBar The ProgressBar view provides visual feedback about some ongoing tasks, such as when u are performing a task in the background.
11	Spinner A drop-down list that allows users to select one value from a set.
12	TimePicker The TimePicker view enables users to select a time of the day, in either 24-hour mode or AM/PM mode.

4.2. Create UI Controls

As explained in previous chapter, a view object may have a unique ID assigned to it which will identify the View uniquely within the tree. The syntax for an ID, inside an XML tag is: `android:id="@+id/text_id"`. To create a UI Control/View/Widget you will have to define a view/widget in the layout file and assign it a unique ID as follows:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
xmlns:android="http://schemas.android.com/apk/res/a
ndroid" android:layout_width="fill_parent"
android:layout_
height="fill_par
ent"
android:orienta
tion="vertical" >
<TextView
android:id="@+id/t
ext_id"
android:layout_wid
th="wrap_content"
android:layout_hei
ght="wrap_content
"    android:text="I
am a TextView" />
```

</LinearLayout>

Then finally create an instance of the Control object and capture it from the layout, use the following:

```
TextView myText = (TextView) findViewById(R.id.text_id);
```

4.3. Android Event Handling

Events are a useful way to collect data about a user's interaction with interactive components of your app, like button presses or screen touch etc. The Android framework maintains an event queue into which events are placed as they occur and then each event is removed from the queue on a first-in, first-out (FIFO) basis. You can capture these events in your program and take appropriate action as per requirements.

There are following three concepts related to Android Event Management:

- **Event Listeners:** The **View** class is mainly involved in building up a Android GUI, same View class provides a number of Event Listeners. The Event Listener is the object that receives notification when an event happens.
- **Event Listeners Registration:** Event Registration is the process by which an Event Handler gets registered with an Event Listener so that the handler is called when the Event Listener fires the event.
- **Event Handlers:** When an event happens and we have registered and event listener for the event, the event listener calls the Event Handlers, which is the method that actually handles the event.

Event Listeners & Event Handlers

Event Handler	Event Listener & Description
onClick()	onClickListener() This is called when the user either clicks or touches or focuses upon any widget like button, text, image etc. You will use onClick() event handler to handle such event.

onLongClick()	onLongClickListener() This is called when the user either clicks or touches or focuses upon any widget like button, text, image etc. for one or more seconds. You will use onLongClick() event handler to handle such event.
onFocusChange()	onFocusChangeListener() This is called when the widget loses its focus i.e. user go away from the view item. You will use onFocusChange() event handler to handle such event.
onKey()	This is called when the user is focused on the item and presses or releases a hardware key on the device. You will use onKey() event handler to handle such event.
onTouch()	onTouchListener() This is called when the user presses the key, releases the key, or any movement gesture on the screen. You will use onTouch() event handler to handle such event.
onMenuItemClick()	onMenuItemClickListener() This is called when the user selects a menu item. You will use onMenuItemClick() event handler to handle such event.

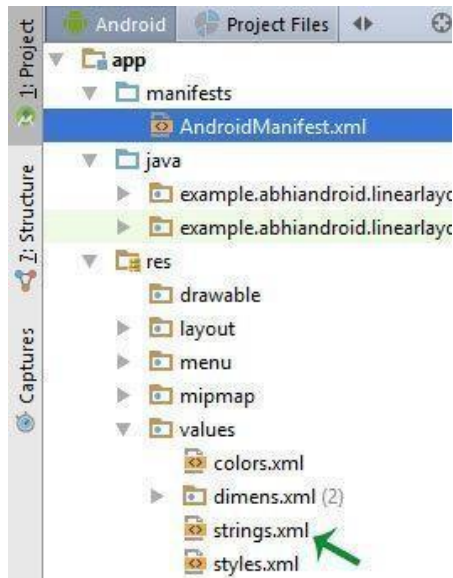
There are many more event listeners available as a part of **View** class like OnHoverListener, OnDragListener etc which may be needed for your application. So I recommend to refer official documentation for Android application development in case you are going to develop a sophisticated apps.

5. Xml Files

5.1. Strings.xml

This xml file is used to replace the Hard-coded strings with a single string. We define all the strings in this xml file and then access them in our app(Activity or in Layout XML files) from this file. This file enhance the reusability of the code.

5.1.1. Location in Android Studio



Below we show **strings.xml** file and define a string in the file.

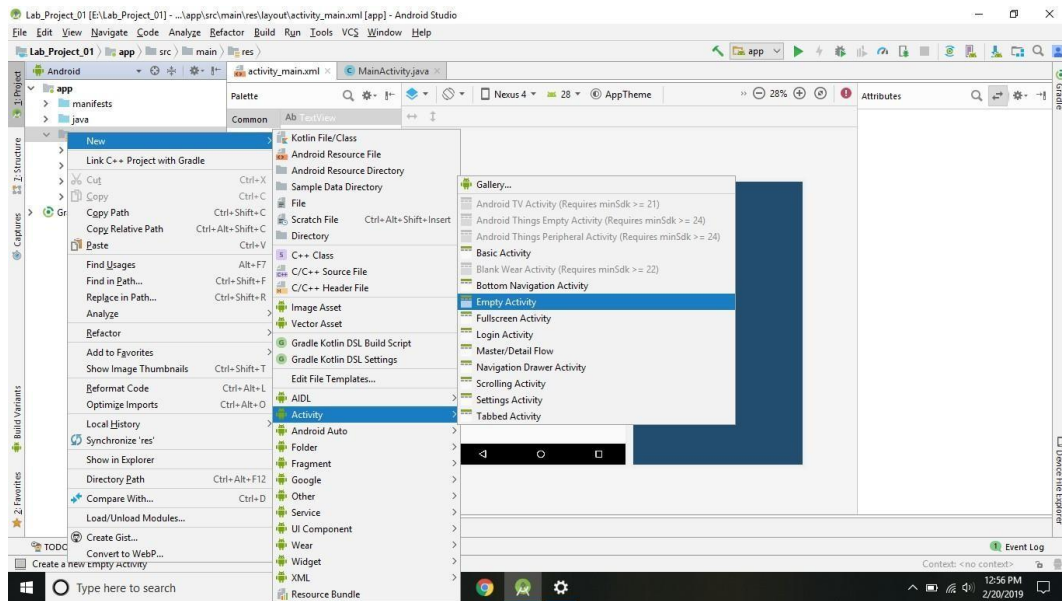
```
<resources>
<string name = "app_name" > My Application </string>
<string name = "hello_world" > Hello world! </string>
<string name = "action_settings" > Settings </string>
<string name = "login" > User Login </string>
<!-- define your strings here -->
</resources>
```

6. Walk through Task

6.1. Creating New Activity

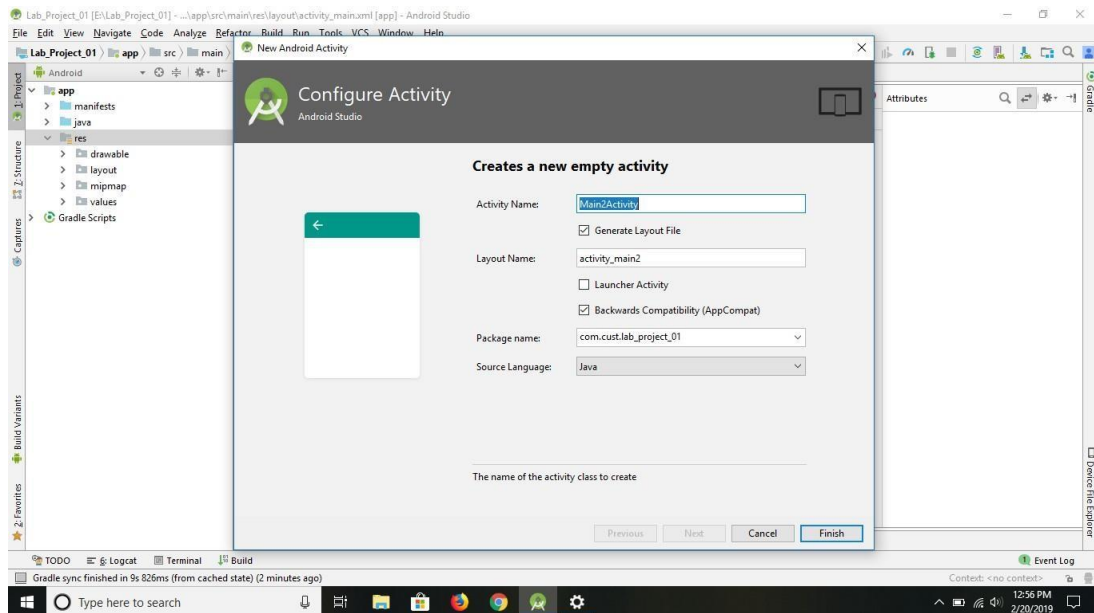
Step 1: Create New Project:

Firstly, open the Android Studio. You will see Welcome to Android Studio on your computer screen. After that you can create new android studio project in Android Studio from “New” then “New Project”.

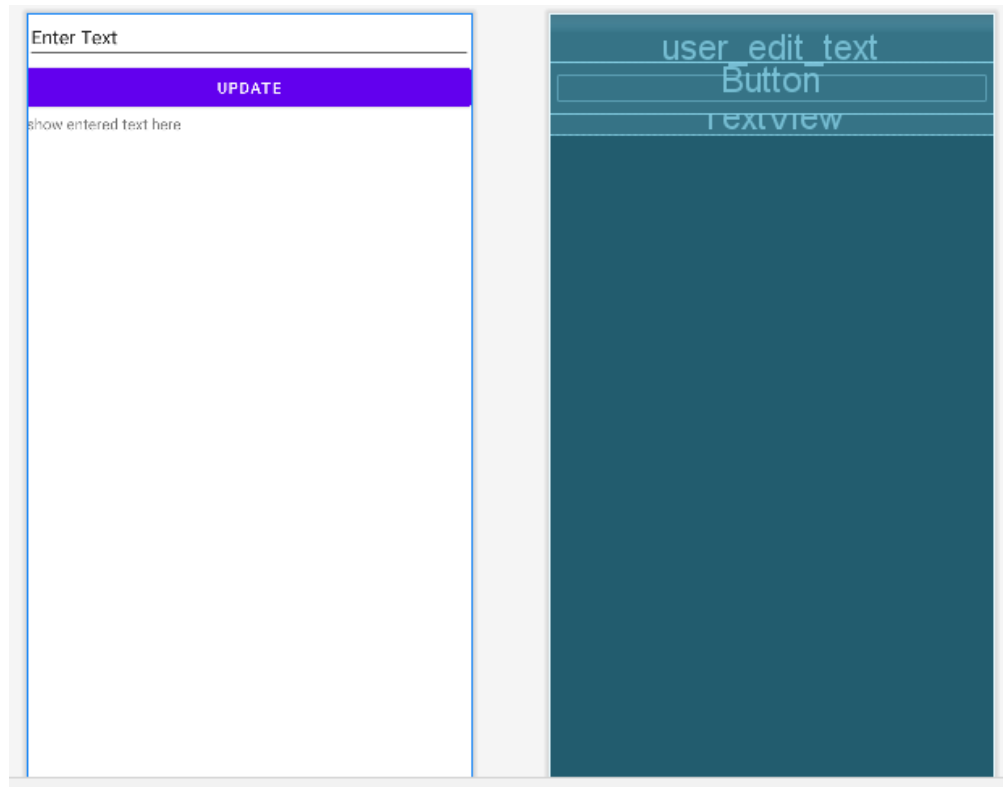


Step 2: Create New Activity

Right click on project package and after that click on the “New” option and then click “New Activity”. Now Select “Empty Activity”. Write name of the activity in “Activity Name”. Click “Finish” button. Wait for the IDE to load and synchronise the new activity in project.



Step 3: Adding Button and Edit Text in Activity:



Step 4: Add View of layout in Activity:

```

1
2
3  import ...
21
22  public class MainActivity extends AppCompatActivity {
23
24      static final String TAG = "MainActivity";
25      @Override
26      protected void onCreate(Bundle savedInstanceState) {
27          super.onCreate(savedInstanceState);
28          setContentView(R.layout.activity_main);
29      }
30  }

```

Step 6: Add Button and Edit Text Reference in Activity:

```

<resources>
    <string name="app_name">My Application</string>
    <string name="string_toast_hello">Hello, you have entered: </string>
</resources>

```

```

private Button showDialogButton;
3 usages
private EditText userEditText;
2 usages
private TextView userTextView;
// declare and define toast text
3 usages
private String toastMessage;
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    // get string from XML
    toastMessage = getString(R.string.string_toast_hello);

    // define UI Component
    showDialogButton = findViewById(R.id.show_dialog_button);
    userEditText = findViewById(R.id.user_edit_text);
    userTextView = findViewById(R.id.user_text_view);
}

```

Step 7: Add onClick Listener.

```

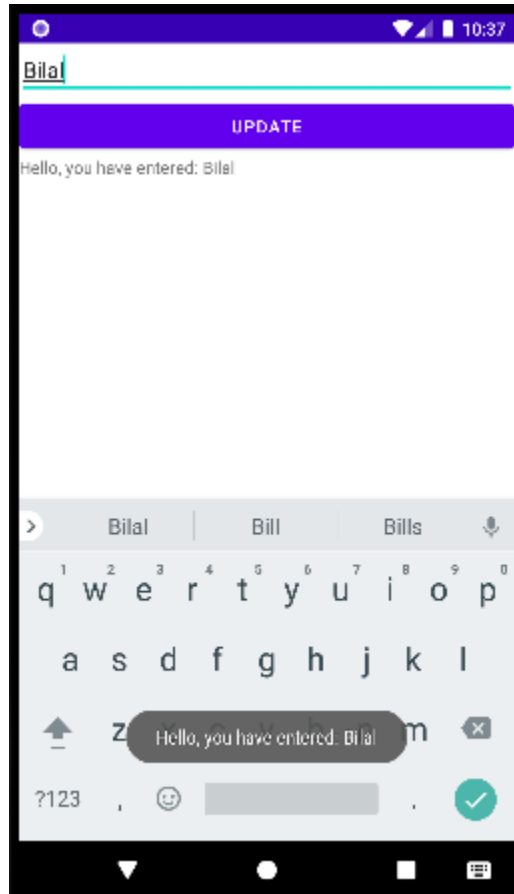
// define UI Component
showDialogButton = findViewById(R.id.show_dialog_button);
userEditText = findViewById(R.id.user_edit_text);
userTextView = findViewById(R.id.user_text_view);

// add button listener
showDialogButton.setOnClickListener(new View.OnClickListener(){
    @Override
    public void onClick(View v) {
        // this function is called when button clicked
        // show toast message
        Toast.makeText(context: MainActivity.this, text: toastMessage + userEditText.getText(),
            Toast.LENGTH_SHORT).show();
        // update text view
        userTextView.setText(toastMessage + userEditText.getText());
    }
});
}

```

Step 8: Run project and enter text and press button.

Click “Run” or press “Shift + F10”.



Practice Tasks

6.2. Practice Task 1

Create an simple “tasbeeh” app what update “tasbee” counter on press of each button press.

